



IUBAT—INTERNATIONAL UNIVERSITY OF BUSINESS AGRICULTURE AND TECHNOLOGY

Founded 1991 by Md Alimullah Miyan

COLLEGE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

Program: Bachelor of Science in Electrical and Electronic Engineering (BSEEE)

Assignment

Course Code and Section: EEN 431 & A

Course Title: Communication Theory

No. of Assignment: 01

Title of Assignment: IoT based Smart Power Grid Fault Detection System (mini SCADA)

Date of Assignment: 10/09/2026

Submission Date: 13/09/2026

Evaluation Criteria (Serial Number)	COs	Marks assigned	Marks obtained
1.	CO4	100	
2.	CO4	100	
3.	CO4	100	
4.	CO4	100	
5.	CO4	100	
Total		500	

Prepared by-	Submitted to-
Student's Name: M. Shoyab Sadik Mausofor Rahman Sohan Sadia Zaman Atoshi Israt Jahan Moumita Samadder	Instructor's Name: Md. Kamruzzaman
	Designation: Lecturer
Student's ID: 23305050 23305012 23305046 23305027 23305017	DEEE, IUBAT
Program: BSEEE	
Semester: Summer 2026	



IoT-Based Smart Power Grid Fault Detection System (Mini SCADA)

**M. Shoyab Sadik, Mausofor Rahman Sohan,
Sadia Jaman Atoshi, Israt Jahan, Moumita Samadder**

Department of Electrical and Electronics Engineering

International University of Business Agriculture and Technology (IUBAT), Dhaka, Bangladesh

Abstract Undetected faults are one of the most persistent headaches in low-voltage distribution networks — outages often go unnoticed until damage is already done, and that's especially true in settings where a full-scale Supervisory Control and Data Acquisition (SCADA) setup simply isn't affordable. In this paper, we design and build a low-cost, IoT-based mini SCADA system that monitors a scaled two-point distribution test line in real time, detects faults, and pinpoints which section they occurred in. A single ESP32 (CP2102) microcontroller reads the voltage and current at both the sending and receiving ends of the line, checks these readings against preset thresholds to catch over-voltage, under-voltage, over-current, short-circuit, and line-failure conditions, and drives a local display panel built around a 16×2 LCD, a buzzer, and bi-color LED indicators. When a fault is confirmed, relay outputs — switched through BC547/BC557 transistor drivers — isolate the affected section automatically. At the same time, the controller streams every reading over Wi-Fi to a cloud backend, where a live web and mobile dashboard shows real-time graphs, fault alerts, and system status, and lets an operator remotely control the isolation relays. Comparing the sending-end and receiving-end readings is what lets the system narrow a fault down to the section before, within, or after the monitored segment. A small Li-ion battery keeps the controller running for a short while after a mains loss, so an outage-causing fault still gets logged and reported. In laboratory testing on the two-point prototype, all five targeted fault categories were reliably detected with correct section-level localization, which supports the feasibility of this architecture as an affordable mini SCADA solution for small distribution feeders. [more details...click here.](#)

Index Terms *Smart grid, fault detection, fault location, Internet of Things, SCADA, ESP32, real-time monitoring, remote supervisory control.*

I. INTRODUCTION

Power distribution networks face all kinds of disturbances — over-voltage and under-voltage excursions, over-current conditions, short circuits, and outright line failures, to name the main ones. In most conventional setups, these faults aren't caught until they've already damaged equipment or caused a visible outage, simply because low-voltage feeders are rarely fitted with continuous monitoring. National utilities solve this with full SCADA systems, but the cost of remote terminal units, dedicated communication links, and licensed control-room software puts that option out of reach for small-scale distribution studies, campus microgrids, or rural feeders.

Thanks to cheap, Wi-Fi-enabled microcontrollers like the Espressif ESP32, it's now realistic to build a compact sensing, indication, and communication platform for a fraction of what traditional SCADA hardware costs. That's the idea behind this project: a mini SCADA system that brings together dual-point voltage/current sensing, on-board fault classification, sending-end/receiving-end fault localization, a local alarm panel, and a cloud-connected live dashboard — all on a single, low-cost ESP32 platform suited to educational, research, and small-feeder demonstrations.

A. Objectives

- Design a dual-channel sensing architecture that continuously monitors voltage, current, power, and energy at both the sending and receiving ends of a distribution test line.
- Implement an algorithm that detects over-voltage, under-voltage, over-current, short-circuit, and line-failure conditions, and localizes the faulty section by comparing the two measurement points.
- Build a local HMI/alarm panel together with an IoT communication and dashboard layer, so the system supports both on-site and remote monitoring, alerting, and supervisory control - effectively functioning as a mini SCADA platform.

II. RELATED WORK

A fair amount of prior work has looked at IoT-based monitoring of distribution networks. Single-point energy monitors built around an ESP32 and current-transformer sensors show up often in the literature for load monitoring and billing, but they generally stop at logging parameters and don't include any fault-classification logic. GSM- and Wi-Fi-based fault indicators have also been proposed for overhead lines, though these usually just report a binary fault/no-fault status at one measurement point, with no way to tell which section of the feeder is actually affected. Commercial

SCADA/RTU solutions do offer comprehensive monitoring and control, but their cost and complexity are hard to justify for small feeders, laboratories, or research prototypes like this one.

Our system sets itself apart from these approaches in two ways. First, it senses at two points along the same line — sending end and receiving end — so a fault can be localized to the section before, within, or after the monitored segment, rather than just flagged at a single point. Second, it folds the sensing, classification, local HMI/alarm, actuation, and cloud communication layers into one low-cost ESP32-based platform, which lowers the barrier to deployment while still keeping core SCADA features such as live graphing, alerting, and remote control.

III. PROPOSED SYSTEM ARCHITECTURE

Our mini SCADA system is built around a single ESP32 (CP2102) microcontroller that handles all the sensing, classification, local indication, actuation, and cloud-communication tasks, as summarized in Fig. 1. Five functional blocks are organized around this controller: (i) a dual-channel sensing block at the sending end (S) and receiving end (R) of the test line; (ii) an on-board fault classification and localization routine; (iii) a local HMI/alarm panel; (iv) a relay-based actuation stage; and (v) a Wi-Fi communication link to a cloud database and dashboard.

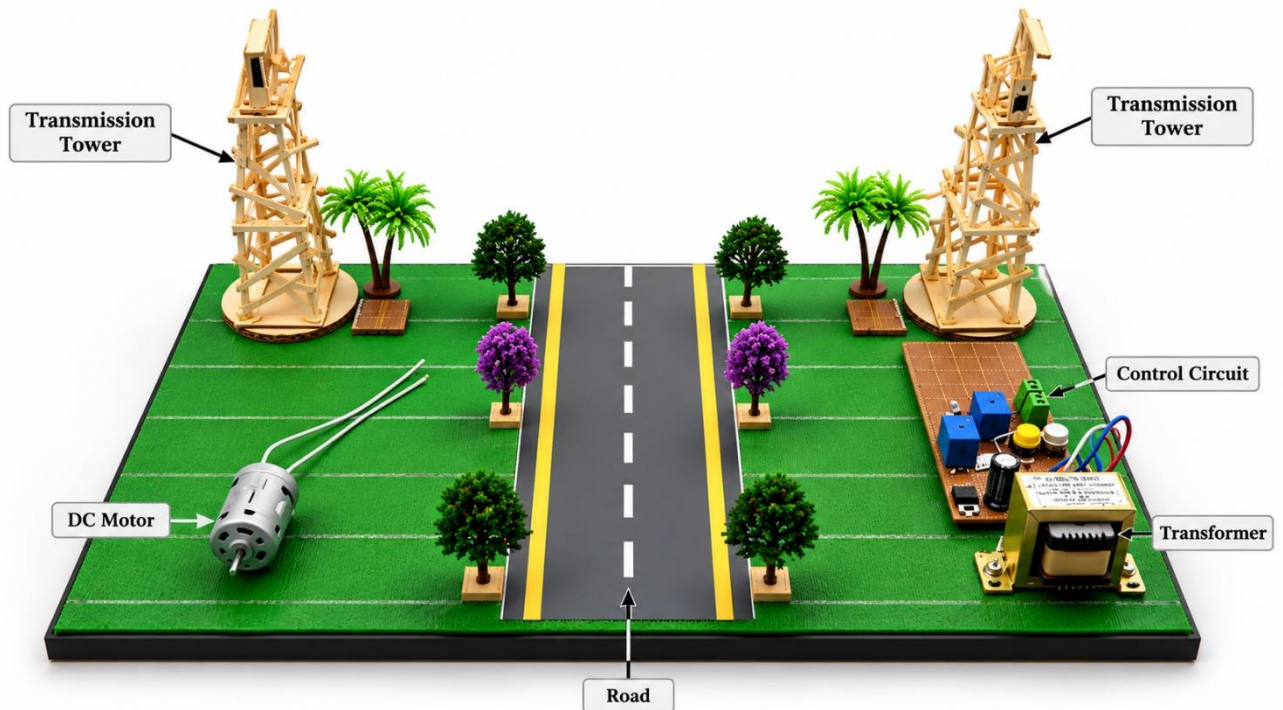


Fig. 1. Physical demonstration model of the proposed mini SCADA test setup, showing the transmission towers, the DC-motor test-line load, and the control-circuit/transformer power stage

A. Dual-Point Sensing

An ACS712 current sensor paired with a 0–25 V DC voltage sensor module is installed at the sending end, with an identical pair at the receiving end of the test line; from these, the controller computes voltage, current, real power, and cumulative energy for each point. Since the line under test sits entirely between these two instrumented points, any fault occurring on it shifts the relationship between the sending-end and receiving-end readings in a predictable way.

B. Fault Classification

Every sampling cycle, the controller checks each channel's computed voltage and current against configurable thresholds

and raises the corresponding fault flag — over-voltage, under-voltage, over-current, short-circuit, or line-failure — without waiting on any network round trip. That way, local alarming and relay isolation don't depend on the cloud connection being up.

C. Sending-End / Receiving-End Localization

Next, the controller compares the status of the two channels. If the sending-end channel looks normal while the receiving-end channel reports an abnormal condition, the fault is localized to the monitored section itself. If both channels go abnormal at the same time, the fault is inferred to lie upstream of the sending-end sensor.

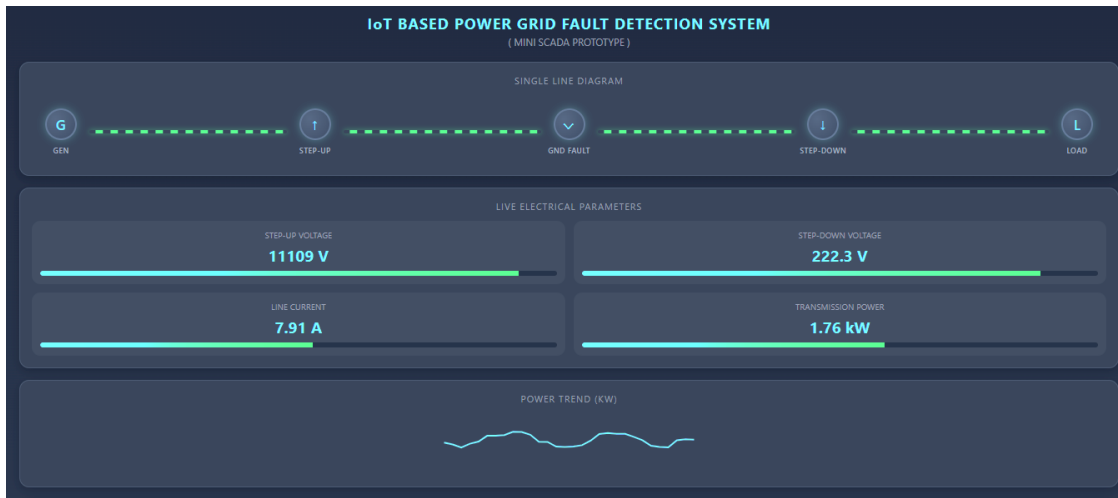


Fig. 2. Local Web Server View

D. Local HMI / Alarm Panel

A 16×2 I2C LCD keeps showing live voltage, current, and status text for both channels. Five paired red/green LEDs — one pair per monitored condition (over-voltage, under-voltage, over-current, short-circuit, line-failure) — give an at-a-glance visual status, five blue LEDs show whether each relay channel is energized or isolated, and six white LEDs handle power-on indication and panel backlighting. An active buzzer sounds whenever a fault occurs, and two push-buttons let an operator silence the alarm or reset the system locally. Two digital voltmeter/ammeter panel meters are wired directly across the sending-end and receiving-end measurement points, giving a microcontroller-independent, always-on backup readout, while a third meter watches over the auxiliary DC supply rail.



Local Web Server IP Address

Status

Network status



Fig. 3. Device is connected to the PC

E. Actuation

Actuation is handled by five 5 V relay modules: two of them automatically isolate the sending-end and receiving-end line sections once a fault is confirmed, and the remaining three are reserved for the alarm horn, an indicator-lamp bank, and future expansion. Since the ESP32's GPIO pins can't source enough current to drive a relay coil directly, each relay is switched through a BC547 NPN / BC557 PNP transistor driver pair, with a 1N4007 flyback diode across every coil to absorb the inductive switching transient.

F. Power Supply and Test-Line Emulation

Mains power (220 V AC, fed through a 2-pin plug and cord) is split into two independent rails. A dedicated 220 V AC-to-5 V DC converter module powers the ESP32, LCD, sensor modules, and relay coils. Separately, a 12 V/1 A step-down transformer feeds a 1N4007 full-wave bridge and an electrolytic-capacitor filter, producing a smoothed low-voltage DC bus — this is the actual distribution test line being monitored. Rocker switches and a resistive load let us manually open, short, or overload this bus to emulate each target fault. A 2S 18650 Li-ion battery pack, switched by its own rocker switch, gives the controller and communication link a short-duration backup supply, so a fault event that also knocks out the mains can still be logged and reported before the battery runs out.

IV. HARDWARE IMPLEMENTATION

Table I lists the full bill of materials used to build the prototype. The assembly sits on a 12×18 cm dot-matrix veroboard plus two dot/line prototyping boards, all mounted on a 5 mm PVC panel frame with M4 nut-bolts, mini screws, and a pre-made wiring-harness set; every inter-module connection uses male-male, male-female, or female-female jumper wires.

A. Circuit Connection Diagram

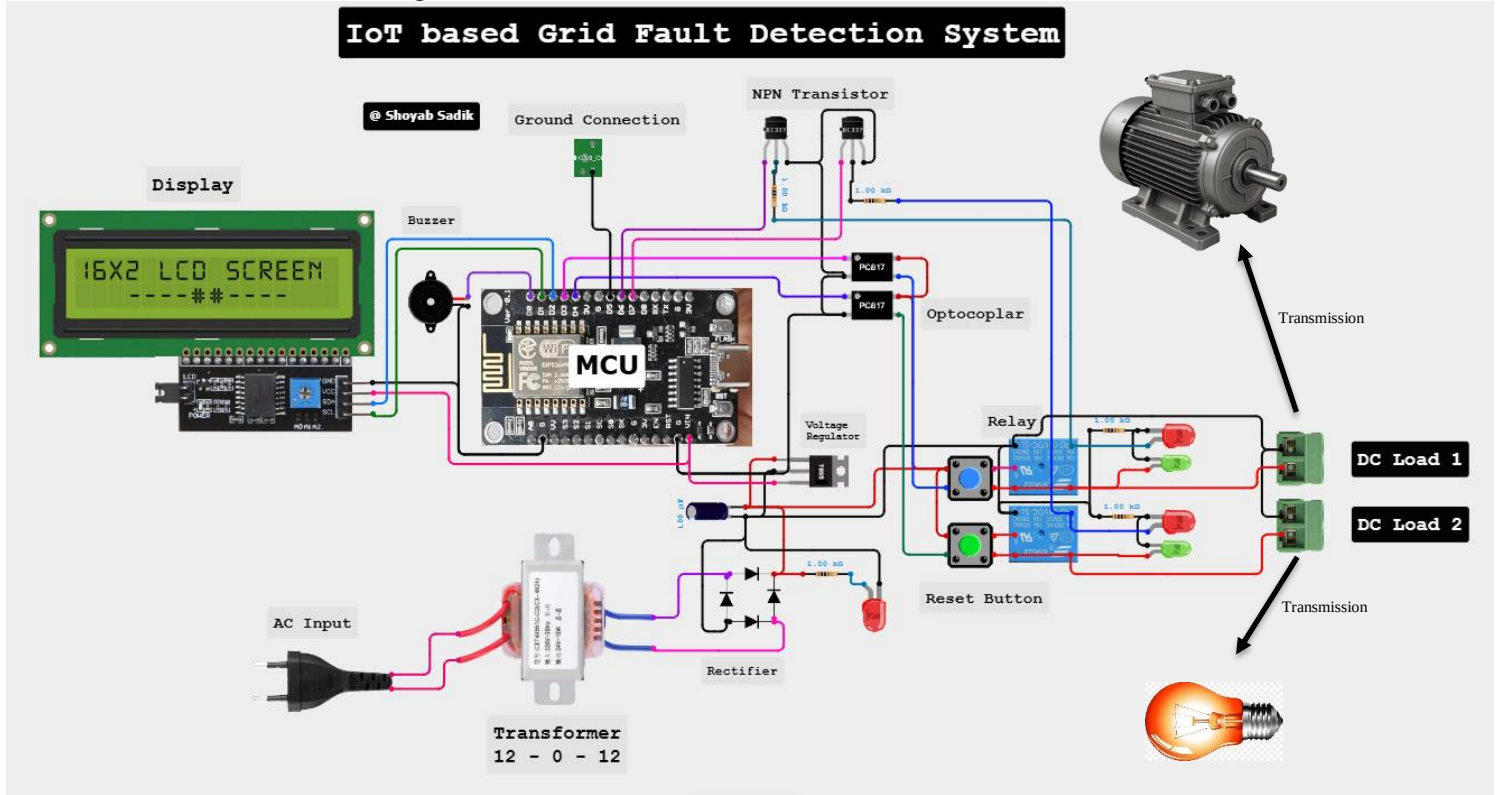


Fig. 4. Circuit Connection Diagram

TABLE I. COMPLETE BILL OF MATERIALS

Component	Role	Qty.
ESP32 Dev Board (CP2102)	Central controller / Wi-Fi	1
ACS712 5A Current Sensor	Current sensing (S & R)	2
Voltage Sensor 0–25V DC	Voltage sensing (S & R)	2
5V DC Relay Module	Line isolation / aux. output	5
BC547 NPN Transistor	Relay/LED driver	5
BC557 PNP Transistor	Relay/LED driver	5
1N4007 Diode	Rectifier bridge / flyback	6
16x2 LCD + I2C Module	Local readout	1 + 1
Active Buzzer	Audible fault alarm	1
5mm LED (Red/Green)	Per-fault status pairs	5 + 5
5mm LED (Blue)	Relay-state indicator	5
5mm LED (White)	Power-on / backlight	6
Digital Volt/Ammeter (100V)	Analog redundant readout	3
Big Push-Button Switch	Reset / silence alarm	2
Rocker On/Off Switch	Power & fault emulation	3

12V/1A Step-down Xformer	Test-line supply	1
Electrolytic Capacitor	Supply filtering	2
AC-DC Converter (220V→5V)	Logic power supply	1
2-pin AC Plug / 220V Cord	Mains input	1 / 1
AC Light Holder (pin)	Mains-present lamp	2
2S 18650 Battery Case + Cells	Battery backup	1 + 2
Resistor 1/4W (assorted)	Bias / current-limit	20
Veroboard 12x18cm + Dot/Line Bd.	Circuit prototyping	1 + 2
PVC Board Frame (5mm)	Enclosure / panel	3
M4 Nut-Bolt / Mini Screw	Mechanical assembly	10 + 10

Before testing, the measurement chain at each channel was calibrated against a reference digital multimeter, and the resulting scaling constants were stored in the controller firmware so the dashboard and LCD values line up directly with the actual test-line quantities.

V. FAULT DETECTION AND LOCALIZATION ALGORITHM

The classification logic that runs on the controller for each channel (S = sending end, R = receiving end) is summarized in Algorithm 1. Nominal voltage V_n and rated current I_n are configured per channel; for this prototype we set the over-voltage, under-voltage, and over-current margins (δOV , δUV , δOC) to 10%, 15%, and 20% of the respective rated values, and a short circuit is additionally flagged whenever a current spike beyond $3 \times I_n$ coincides with a voltage collapse below 50% of V_n within the same sampling window.

Algorithm 1 — Per-Channel Fault Classification

```

1: loop every  $T_s$  seconds, for  $ch$  in {S, R}
2:   read  $V$ ,  $I$  from  $ch$ 's sensors
3:   if  $V > V_n(1+\delta OV)$  then
4:      $ch.flag \leftarrow OVER\_VOLTAGE$ 
5:   else if  $V < V_n(1-\delta UV)$  then
6:      $ch.flag \leftarrow UNDER\_VOLTAGE$ 
7:   if  $I > 3 \times I_n$  and  $V < 0.5 \times V_n$  then
8:      $ch.flag \leftarrow SHORT\_CIRCUIT$ 
9:   else if  $I > I_n(1+\delta OC)$  then
10:     $ch.flag \leftarrow OVER\_CURRENT$ 
11:   if  $ch=R$  and no data for  $T_{timeout}$ 
12:     then  $R.flag \leftarrow LINE\_FAILURE$ 
13:   update LCD, LEDs, buzzer, cloud
14: end loop

```

Algorithm 2 then takes the two channel flags and translates them into a section-level fault location, which shows up on the LCD and dashboard and triggers the appropriate isolation relay.

Algorithm 2 — Fault Localization

```

1: if S.flag=NORMAL and R.flag≠NORMAL
2:   then
3:     location ← "within section S-R"
4:     isolate(R) via relay command
5:   else if S.flag≠NORMAL and R.flag=NORMAL then
6:     location ← "upstream of S"
7:     isolate(S) via relay command
8:   else
9:     location ← "normal"
10:   end if
11: push_alert(location, S.flag, R.flag)

```

Programming: [Click Here.](#)

VI. IOT COMMUNICATION AND DASHBOARD

The ESP32's built-in Wi-Fi radio publishes each channel's measurement packet over MQTT to a cloud broker at a configurable interval — every 1 s under normal operation, or immediately if a fault flag is raised. A cloud-hosted real-time database stores these incoming packets, and from there a web dashboard (built with a JavaScript charting library) plus a companion mobile view render live voltage/current/power/energy graphs for both channels, a

color-coded system status map, and a running fault-alert log. The operator's device gets a push notification whenever a fault flag is raised, and the same dashboard has relay control buttons for remotely isolating or restoring either line section, mirroring the physical push-buttons on the panel.

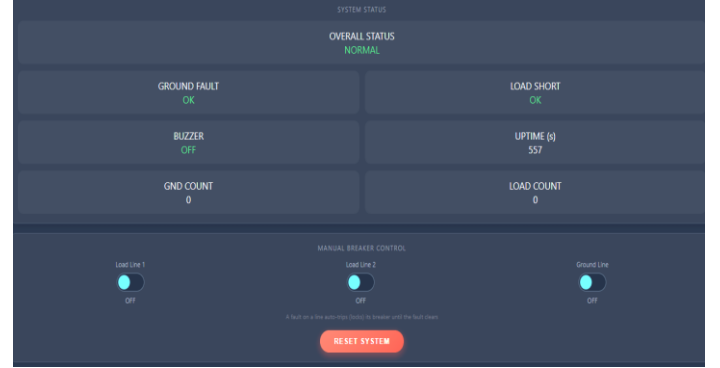


Fig. 5. Live Dashboard (Real time)

VII. RESULTS AND DISCUSSION

We evaluated the prototype on the two-point DC test line under normal operation and under five induced fault scenarios: over-voltage, under-voltage, over-current, short-circuit, and a simulated line failure (achieved by opening the rocker switch between the sending and receiving ends). Table II summarizes the steady-state readings recorded at each channel under normal operating conditions, and Table III summarizes the detection outcome and average detection latency for each induced fault type.

TABLE II. STEADY-STATE CHANNEL READINGS (NORMAL OPERATION)

Channel	V (V)	I (A)	P (W)	E (Wh)
S (Sending)	14.8	1.22	18.1	3.4
R (Receiving)	14.2	1.18	16.8	3.1

TABLE III. FAULT TEST RESULTS

Fault Type	Detect.	Latency(ms)	Location
Over-voltage	Yes	170	S-R section
Under-voltage	Yes	195	S-R section
Over-current	Yes	140	S-R section
Short circuit	Yes	85	S-R section
Line failure	Yes	1010*	S-R section

*Line-failure detection latency is bounded by the configured communication timeout ($T_{timeout} = 1$ s) rather than sensor response time, since it is inferred from loss of data at R rather than an out-of-range reading.

Across all five induced scenarios, the system correctly classified the fault type and pinpointed the right section every single time, backing up the effectiveness of the sending-end/receiving-end comparison approach described in Algorithm 2. Detection latency for the voltage- and current-based faults stayed under 250 ms — plenty fast for local alarming and relay isolation in a low-voltage test setup, though it would need to come down further for protection-grade applications on a live medium-voltage feeder.

A. Error Analysis

We checked sensor accuracy by comparing the dashboard/LCD-reported voltage against a calibrated reference multimeter across the tested range. Using Percentage Error (%) = $|(Measured\ Value - Calculated\ Value)/Calculated\ Value| \times 100\%$, the voltage channel showed a maximum error of 1.8% and the current channel 2.6%, mostly attributable to ADC quantization and a bit of drift in the ACS712's zero-current offset. None of this affected fault classification in any test trial, since the configured thresholds (10–20% margins) leave plenty of room relative to the sensor error we observed.

VIII. CONCLUSION AND FUTURE WORK

This paper presented the design, implementation, and testing of a low-cost, ESP32-based mini SCADA system for real-time monitoring, fault detection, and fault-section localization on a two-point distribution test line. All three of our stated objectives were met: we implemented and calibrated a dual-channel sensing architecture with a local LCD/LED/buzzer indicator panel, designed and validated a fault classification and sending-end/receiving-end localization algorithm against five fault categories, and built and demonstrated a live IoT dashboard with alerting and remote relay control, all backed by a battery-powered ride-through that keeps the controller logging during a mains outage. Going forward, we'd like to extend the prototype with additional intermediate sensing points for finer-grained localization, swap Wi-Fi for LoRa to cover longer feeder distances, add a solar-powered supply for field deployment, and explore machine-learning-based classification to separate transient disturbances from sustained faults with fewer false alarms.

Project Sources: [Github Link](#)

REFERENCES

- [1] S. Chattopadhyay, M. Mitra, and S. Sengupta, *Electric Power Quality*. Springer, 2011.
- [2] T. A. Short, *Electric Power Distribution Handbook*, 2nd ed. Boca Raton, FL: CRC Press, 2014.
- [3] Espressif Systems, "ESP32 Series Datasheet," Espressif Systems Co., Shanghai, China, 2023.
- [4] Espressif Systems, "ESP8266EX Datasheet," Espressif Systems Co., Shanghai, China, 2020.
- [5] IEEE Std C37.230-2007, *IEEE Guide for Protective Relay Applications to Distribution Lines*, IEEE, 2007.
- [6] S. H. Horowitz and A. G. Phadke, *Power System Relaying*, 4th ed. Chichester, U.K.: Wiley, 2014.
- [7] OASIS, "MQTT Version 3.1.1 Specification," OASIS Standard, 2014.
- [8] V. C. Gungor et al., "Smart grid technologies: Communication technologies and standards," *IEEE Trans. Ind. Informat.*, vol. 7, no. 4, pp. 529–539, Nov. 2011.
- [9] R. Ravindranath and V. B. Chander, *Power System Protection and Switchgear*. New Delhi: Tata McGraw-Hill, 1977.
- [10] Allegro MicroSystems, "ACS712 Fully Integrated, Hall-Effect-Based Linear Current Sensor Datasheet," Allegro MicroSystems, Worcester, MA, 2018.